

Architecture technique Frontend — JB_STUDIOS_FIRST_SHOT

Découpage détaillé module par module (Nuxt.js 3 / Vue 3)

Vue d'ensemble

Le frontend est architecturé en **12 modules indépendants mais interconnectés**, organisés du plus fondamental (configuration) au plus périphérique (déploiement). Chaque module a une responsabilité unique, un emplacement précis dans l'arborescence, et des dépendances explicites vers les modules qu'il consomme.

1. Fondations & configuration
2. Design System / UI Kit
3. Layouts & Navigation
4. Routing & Pages
5. Sections de contenu
6. Galeries média (photo/vidéo)
7. Portfolio dynamique & Blog (contenu)
8. Formulaires & validation
9. État global & composables
10. Animations & transitions
11. Intégrations tierces
12. SEO, performance & qualité

Module 1 — Fondations & configuration

Objectif : poser le socle technique du projet : configuration Nuxt, TypeScript, alias, variables d'environnement, modules officiels.

Emplacement : racine du projet (`nuxt.config.ts`, `tsconfig.json`, `.env`).

Dépendances (packages) :

- `nuxt` (3.x)
- `@nuxtjs/tailwindcss`
- `@vueuse/nuxt`
- `@nuxt/image`
- `@nuxt/eslint`

Détails d'implémentation :

- `nuxt.config.ts` centralise : liste des `modules`, `runtimeConfig` (clés API publiques/privées séparées — ex. clé Google Maps publique vs clé Resend privée côté serveur uniquement), `app.head` par défaut (favicon, lang="fr"), et les cibles de build (`nitro.preset`) selon l'hébergeur choisi : `vercel` ou `netlify`).
- `runtimeConfig` distingue explicitement :
 - `runtimeConfig.public` → exposé au navigateur (clé Google Maps, ID Google Analytics)
 - `runtimeConfig` (racine) → serveur uniquement (clé API Resend, secret reCAPTCHA)
- Alias de chemins configurés (`~/components`, `~/composables`, `~/types`) pour des imports courts et lisibles.
- Fichier `.env` (non versionné) + `.env.example` documenté pour l'onboarding d'un futur développeur.
- TypeScript en mode strict (`"strict": true`) dès le départ pour fiabiliser les échanges de données entre modules (formulaires, API, contenu).

Points d'attention :

- Ne jamais exposer une clé sensible dans `runtimeConfig.public`.
 - Le choix du `nitro.preset` doit être fixé avant la Phase 3 (développement) pour éviter des surprises de compatibilité API à la fin du projet.
-

Module 2 — Design System / UI Kit

Objectif : traduire le design system (palette navy/or, typographies, espacements) en composants réutilisables, indépendants de toute page métier.

Emplacement : `components/ui/`, `assets/css/`, `tailwind.config.ts`.

Dépendances :

- `@nuxtjs/tailwindcss`
- `@nuxt/ui` (ou `shadcn-vue` en alternative)

Détails d'implémentation :

- `tailwind.config.ts` étend le thème par défaut avec les tokens de marque : couleurs (`navy`, `gold`, `ivory`, `slate`), polices (`font-serif` → Playfair Display / Fraunces, `font-sans` → Inter/Manrope), échelle d'espacement en base 8.
- Composants atomiques créés une fois pour toutes : `UIButton.vue` (variantes `primary` / `secondary` / `whatsapp`), `UiCard.vue`, `UiBadge.vue`, `UiInput.vue`, `UiAccordion.vue`, `UiModal.vue`.

- Chaque composant expose des **props typées** (`variant`, `size`, `disabled`) plutôt que des classes Tailwind librement passées, pour garantir la cohérence visuelle sur tout le site.
- Utilisation des **slots nommés** Vue pour permettre la personnalisation du contenu interne (ex. `<UiButton><template #icon>...</template></UiButton>`) sans dupliquer le composant.
- Un fichier (`assets/css/main.css`) centralise les styles globaux non couverts par Tailwind (ex. sélection de texte, scrollbar personnalisée).

Points d'attention :

- Ce module ne doit avoir **aucune dépendance vers les pages ou sections métier** — il doit rester testable et réutilisable de façon isolée.
 - Documenter chaque composant (props/slots) dans un commentaire d'en-tête pour faciliter la maintenance future.
-

Module 3 — Layouts & Navigation

Objectif : définir la structure globale des pages (header, footer, menu) et les variantes de mise en page.

Emplacement : `layouts/`, `components/layout/`.

Dépendances : Module 2 (UI Kit), Module 10 (animations pour le menu mobile).

Détails d'implémentation :

- `layouts/default.vue` : structure standard avec `<AppHeader />`, `<slot />` (contenu de page), `<AppFooter />`.
- `layouts/legal.vue` : layout épuré sans CTA marketing, utilisé pour Mentions légales / Confidentialité.
- `AppHeader.vue` : header **sticky** avec effet de fond transparent → opaque au scroll (détecté via `useScroll` de VueUse), CTA « Devis » toujours visible, bouton menu mobile.
- `AppMobileMenu.vue` : menu plein écran animé (GSAP timeline), fermeture au clic sur un lien ou sur l'overlay, verrouillage du scroll du `body` pendant l'ouverture (`useScrollLock` de VueUse).
- `AppFooter.vue` : liens de navigation secondaire, réseaux sociaux, mentions légales, coordonnées.
- Le choix du layout par page se fait via `definePageMeta({ layout: 'legal' })` dans la page concernée.

Points d'attention :

- Le header doit rester accessible au clavier (focus visible, `aria-expanded` sur le bouton menu mobile).
-

Module 4 — Routing & Pages

Objectif : organiser les routes du site selon l'arborescence validée dans le cahier des charges, y compris les routes dynamiques.

Emplacement : `pages/`.

Dépendances : Module 3 (layouts), Module 5 (sections), Module 7 (contenu dynamique).

Détails d'implémentation :

- Routage basé sur le système de fichiers de Nuxt (`file-based routing`) : chaque fichier `.vue` dans `pages/` devient une route automatiquement.
- Routes dynamiques pour le contenu variable : `pages/portfolio/[slug].vue` et `pages/blog/[slug].vue`, alimentées par le Module 7.
- `definePageMeta()` utilisé sur chaque page pour déclarer : le layout, les métadonnées SEO par défaut (relayées au Module 12), et d'éventuelles règles de rendu (`ssr: true` par défaut, exceptions si besoin).
- Un middleware léger (`middleware/redirects.global.ts`) gère les redirections d'anciennes URLs si nécessaire après la mise en ligne.
- Page `pages/devis.vue` isolée du reste (formulaire multi-étapes, Module 8) pour permettre un tracking de conversion dédié (Module 12 / Analytics).

Points d'attention :

- Garder une correspondance stricte entre l'arborescence validée dans le cahier des charges et les routes réelles, pour ne pas casser le plan SEO prévu.
-

Module 5 — Sections de contenu

Objectif : fournir les blocs visuels réutilisables qui composent chaque page (hero, grille de services, témoignages, etc.), assemblés page par page.

Emplacement : `components/sections/`.

Dépendances : Module 2 (UI Kit), Module 10 (animations), Module 6 (pour les extraits de galerie).

Détails d'implémentation :

- Composants « section » un par bloc identifié dans le cahier des charges :
`HeroHome.vue`, `ServicesGrid.vue`, `ValuesBlock.vue`,
`TestimonialsCarousel.vue`, `TrustBar.vue` (chiffres clés), `CtaBanner.vue`,
`ContactQuickBlock.vue`.
- Chaque section reçoit ses données via **props typées** (interface TypeScript dédiée), jamais de données codées en dur dans le composant — les données proviennent soit d'un fichier de configuration (`content/site-config.ts`), soit du CMS (Module 7).
- `HeroHome.vue` gère spécifiquement le fond vidéo : lecture automatique silencieuse (`muted`, `playsinline`), poster image affiché avant chargement, et bascule vers une image statique sur connexion lente (détectée via `navigator.connection` quand disponible).
- `TestimonialsCarousel.vue` implémenté avec un composant de carrousel léger (ex. `embla-carousel-vue`) plutôt qu'une librairie lourde, pour limiter le poids du bundle.

Points d'attention :

- Les sections doivent rester « bêtes » (dumb components) : assemblage visuel uniquement, aucune logique métier ou appel réseau direct.

Module 6 — Galeries média (photo / vidéo)

Objectif : afficher, filtrer et faire vivre les galeries photo et vidéo — cœur de la conversion du site.

Emplacement : `components/gallery/`, `composables/useGallery.ts`.

Dépendances : Module 2 (UI Kit), Module 9 (état/composables), `@nuxt/image`, Cloudinary.

Détails d'implémentation :

- `GalleryGrid.vue` : grille de type masonry (via CSS `columns`) natif ou librairie légère type `vue-masonry-wall`), alimentée par un tableau d'items typés (`{ id, category, src, alt }`).
- `GalleryFilters.vue` : boutons de filtre par catégorie ; le filtrage se fait **côté client** sans rechargement, avec transition animée (`<TransitionGroup>` de Vue) lors du réordonnancement des items.
- `GalleryLightbox.vue` : composant modal plein écran, navigation clavier (flèches, `Échap`), swipe tactile sur mobile, chargement de l'image en haute résolution uniquement à l'ouverture (l'image de grille reste en basse résolution).
- Toutes les images passent par `<NuxtImg>` (module `@nuxt/image`) avec `format="webp"`, `loading="lazy"` par défaut sauf pour l'image `hero` (chargée en priorité).

- `VideoGrid.vue` et `VideoPlayer.vue` : les vidéos ne sont **jamais auto-hébergées** — embed via l'API Vimeo/YouTube (Module 11), avec miniature personnalisée affichée tant que la vidéo n'est pas lancée (évite le chargement du lecteur tant qu'il n'est pas nécessaire).
- `composables/useGallery.ts` centralise la logique de filtrage/pagination pour être réutilisée entre la page Galerie Photos, Galerie Vidéos et les extraits affichés en page d'Accueil.

Points d'attention :

- C'est le module le plus sensible en performance : chaque décision (format d'image, lazy loading, lightbox) impacte directement le Core Web Vitals (LCP notamment si mal géré sur le hero).

Module 7 — Portfolio dynamique & Blog (contenu)

Objectif : gérer le contenu éditorial mis à jour régulièrement par l'équipe du studio (études de cas, articles de blog), sans intervention technique.

Emplacement : `content/` (si Nuxt Content) ou appels API vers un CMS externe, `composables/useContent.ts`.

Dépendances : Module 4 (routes dynamiques), Nuxt Content ou Sanity/Strapi.

Détails d'implémentation (option retenue : Nuxt Content) :

- Chaque projet de portfolio et chaque article de blog est un fichier Markdown avec frontmatter (`content/portfolio/mariage-x.md`, `content/blog/choisir-photographe-mariage.md`).
- `queryContent()` (API Nuxt Content) utilisé dans `pages/portfolio/[slug].vue` et `pages/blog/[slug].vue` pour récupérer le contenu correspondant au `slug` de l'URL, avec gestion du cas « non trouvé » (`createError({ statusCode: 404 })`).
- Listes (page d'index portfolio/blog) générées via `queryContent().sort().find()`, avec tri par date et filtrage par catégorie/tag défini en frontmatter.
- **Alternative si volume de contenu important ou besoin d'une interface d'administration plus riche pour l'équipe non technique :** bascule vers Sanity.io — dans ce cas, ce module devient un ensemble de fonctions `composables/useSanity.ts` interrogeant l'API Sanity via GROQ, avec le même contrat de données côté composants (aucun changement requis dans les Modules 4 et 5).

Points d'attention :

- Le contrat de données (forme des objets `PortfolioItem`, `BlogArticle`) doit être défini en TypeScript **avant** le choix définitif CMS interne/externe, pour que le reste du

frontend soit indépendant de cette décision.

Module 8 — Formulaires & validation

Objectif : fiabiliser la collecte des demandes de devis et messages de contact — module directement lié à la conversion.

Emplacement : `components/forms/`, `server/api/`, `composables/useDevisForm.ts`.

Dépendances : Module 1 (config/env), Module 9 (état), Nitro (Module backend), reCAPTCHA.

Détails d'implémentation :

- Schémas de validation définis avec **Zod**, partagés entre le client (validation immédiate) et le serveur (validation de sécurité, ne jamais faire confiance au seul client).
- `ContactForm.vue` : formulaire simple à validation en temps réel (`@blur` par champ), retour visuel immédiat (bordure rouge/verte, message d'erreur sous le champ).
- `DevisForm.vue` : formulaire **multi-étapes** géré par un composable dédié `useDevisForm.ts` qui maintient l'état courant (`currentStep`, `formData`) en mémoire réactive (`reactive()`), avec une barre de progression pilotée par `currentStep / totalSteps`.
- Chaque étape est un sous-composant (`DevisStepEvent.vue`, `DevisStepBudget.vue`, `DevisStepContact.vue`, `DevisStepSummary.vue`), monté conditionnellement — pas de rechargement de page entre les étapes.
- Soumission finale : appel `$fetch('/api/devis', { method: 'POST', body })` vers la route Nitro correspondante, qui valide à nouveau les données (Zod côté serveur), vérifie le token reCAPTCHA, déclenche l'envoi d'e-mail (Resend/SendGrid) et enregistre la demande en base (Supabase/Firebase).
- Gestion d'état de soumission claire : `idle` → `submitting` → `success` / `error`, avec composant de confirmation dédié (`FormSuccess.vue`) plutôt qu'une simple alerte.

Points d'attention :

- Toujours revalider côté serveur (Nitro), même si le client valide déjà — le client peut être contourné.
 - Prévoir un honeypot (champ caché) en complément du reCAPTCHA pour filtrer les bots basiques sans friction utilisateur.
-

Module 9 — État global & composables

Objectif : centraliser la logique et l'état partagés entre plusieurs modules (ouverture de modales, filtres actifs, panier de sélection de prestations, etc.).

Emplacement : `composables/`, éventuellement `stores/` si Pinia est introduit.

Dépendances : transverse — consommé par la majorité des autres modules.

Détails d'implémentation :

- Pour un site vitrine de cette taille, l'état global reste **volontairement léger** : usage de composables Vue natifs (`useState()` de Nuxt, qui est SSR-safe) plutôt que d'installer Pinia dès le départ.
- Composables clés : `useDevisForm.ts` (Module 8), `useGallery.ts` (Module 6), `useModal.ts` (gestion générique d'ouverture/fermeture de modales, réutilisé par la lightbox et les popups), `useWhatsapp.ts` (génère le lien `wa.me` avec message pré-rempli selon la page courante).
- `useState()` est préféré à des `ref()` globaux exportés, car il évite les fuites d'état entre requêtes différentes en SSR (chaque requête serveur obtient son propre état).
- Si le projet évolue vers un espace client (roadmap future), **Pinia** sera introduit à ce moment-là, sans remise en cause de l'architecture existante.

Points d'attention :

- Ne pas globaliser un état qui pourrait rester local à un composant — la simplicité prime tant que la complexité ne l'exige pas.
-

Module 10 — Animations & transitions

Objectif : implémenter le langage d'animation défini dans le design system (fade-in au scroll, parallax léger, transitions de page) de façon cohérente et performante.

Emplacement : `composables/useScrollReveal.ts`, `plugins/gsap.client.ts`, transitions dans `app.vue`.

Dépendances : GSAP + ScrollTrigger, `@vueuse/nuxt`.

Détails d'implémentation :

- GSAP chargé uniquement côté client via un plugin `.client.ts` (évite tout conflit avec le rendu SSR).
- `useScrollReveal.ts` : composable générique appliqué via une directive personnalisée (`v-reveal`) sur les éléments qui doivent apparaître au scroll — évite de dupliquer la configuration GSAP dans chaque composant de section.

- Transitions de page définies globalement dans `app.vue` via `<NuxtPage :transition="{ name: 'page', mode: 'out-in' }" />`, avec les classes CSS correspondantes (`page-enter-active`, `page-leave-active`) pour un fondu enchaîné cohérent entre toutes les pages.
- Le menu mobile (Module 3) et la lightbox (Module 6) utilisent des timelines GSAP dédiées, orchestrées séparément pour rester performantes (pas de ScrollTrigger inutile sur des éléments toujours visibles).
- Respect de `prefers-reduced-motion` : toutes les animations sont désactivées ou réduites si l'utilisateur a activé cette préférence système (accessibilité).

Points d'attention :

- Chaque animation doit être justifiée par une intention UX (guider l'œil, confirmer une action) — jamais de mouvement gratuit qui nuirait à la performance perçue.
-

Module 11 — Intégrations tierces

Objectif : connecter le frontend aux services externes nécessaires (carte, réseaux sociaux, messagerie, vidéo) sans alourdir le site.

Emplacement : `components/integrations/`, `composables/usewhatsapp.ts` (partagé avec Module 9).

Dépendances : clés API définies au Module 1.

Détails d'implémentation :

- **Google Maps :** intégration via une iframe légère (pas de SDK JavaScript complet) chargée en `loading="lazy"`, ou composant `MapEmbed.vue` avec chargement différé au scroll (`useIntersectionObserver` de VueUse) pour ne pas pénaliser le temps de chargement initial de la page Contact.
- **WhatsApp :** bouton flottant global (`WhatsappButton.vue`) dans le layout par défaut) générant un lien `https://wa.me/<numéro>?text=<message>`, le message étant contextualisé selon la page (`useRoute()`) via le composable `usewhatsapp.ts`.
- **Instagram :** affichage du flux via l'API Instagram Graph, appelée côté serveur (route Nitro `server/api/instagram.get.ts`) pour ne pas exposer le token d'accès côté client, avec mise en cache de quelques heures pour limiter les appels API.
- **YouTube / Vimeo :** embed différé — seule la miniature est chargée initialement (Module 6), l'iframe du lecteur n'est injectée dans le DOM qu'au clic de l'utilisateur (facteur de performance important).
- **TikTok / Facebook :** simples liens sortants dans le footer, sans embed lourd.

Points d'attention :

- Toute intégration tierce est chargée de façon **différée et isolée**, jamais dans le bundle JavaScript principal, pour préserver le temps de chargement initial.
-

Module 12 — SEO, performance & qualité

Objectif : garantir que chaque page est correctement indexable, rapide, et conforme aux standards définis dans le cahier des charges.

Emplacement : transverse (`useSeoMeta()` dans chaque page), `nuxt.config.ts` (modules SEO), CI.

Dépendances : `@nuxtjs/sitemap`, `@nuxtjs/robots`, `nuxt-schema-org`, ESLint, Lighthouse CI.

Détails d'implémentation :

- Chaque page définit ses métadonnées via `useSeoMeta()` (title, description, OG image, canonical) directement dans son fichier `pages/*.vue`, en s'appuyant sur les données du Module 7 pour les pages dynamiques (portfolio, blog).
- `@nuxtjs/sitemap` génère automatiquement le `sitemap.xml` à partir des routes statiques et dynamiques (y compris les slugs de portfolio/blog).
- `nuxt-schema-org` injecte les données structurées (`LocalBusiness`, `FAQPage`, `VideoObject`, `Review`) définies dans le cahier des charges, module par module (ex. le composant FAQ injecte automatiquement son propre balisage `FAQPage`).
- Budget de performance suivi via **Lighthouse CI**, intégré au pipeline de déploiement : le build échoue si le score de performance descend sous un seuil défini (ex. 85/100 sur la page d'accueil).
- ESLint + règles Vue/Nuxt officielles appliquées en pre-commit (`husky` + `lint-staged`) pour maintenir la qualité de code sur la durée du projet.

Points d'attention :

- Le SEO n'est pas un module « à la fin » : chaque module de contenu (5, 6, 7) doit produire des données déjà structurées pour que ce module n'ait qu'à les exposer, sans logique métier supplémentaire.
-

Séquence de développement recommandée

L'ordre ci-dessus correspond aussi à l'ordre de développement conseillé : les modules 1 et 2 doivent être stabilisés avant tout le reste (ils conditionnent tous les autres), les modules 3 et 4 posent le squelette navigable, les modules 5 à 8 constituent le cœur fonctionnel visible par l'utilisateur, et les modules 9 à 12 sont transverses — introduits progressivement dès

qu'un premier module en a besoin, puis consolidés en fin de développement (Phase 6 — Tests & QA du planning).